

정소원

Sowon Jung

010-9349-1709 swj960515@gmail.com

PROJECT PORTFOLIO

github.com/ss-won velog.io/@ss-won linkedin.com/in/sowon-jung-573867193

01 /

파이레테크놀로지
2024.09 — 현재

STACK

- TypeScript
- React
- Next.js
- TailwindCSS
- TanStack Query
- wagmi · viem
- Sentry

프론트엔드 성능 병목 개선

Gas Top-up · BiFi WebView · Bifrost Explorer

Gas Top-up DApp(Pockie 지갑 가스 충전)·BiFi WebView(DeFi 렌딩)·Bifrost Explorer(Bifrost Network 블록 탐색기)에서 발생한 성능 병목을 서비스별 원인에 맞춰 각각 개선했습니다.

Gas Top-up — 호출 구조 단순화

- 보유 자산 수에 비례하던 자산·잔액 조회를 **지원 토큰 기준 조회**로 정리
- 사내 SDK가 wallet 연결 시 asset 정보를 최상위 provider에 저장하는 구조를 파악하고, 하위 앱 단위에서 **중복 설정된 asset 조회를 제거**해 진입 시 호출 단순화

BiFi WebView — bridge pair 조회 개선

- 토큰·네트워크 선택마다 반복 조회하던 구조(**1+N회**)를 outbound/inbound **2회** 캐시 조회로 전환

Bifrost Explorer — 목록 렌더링 최적화

- 가상화 도입을 검토했으나 행 간 whitespace 이슈로 단계적 렌더링으로 전환 — 초기 렌더링 범위 제한·WebSocket **30초 배치 처리** 적용

Route	LCP (before → after)	TBT (before → after)	개선
/blocks	3.7s 1.9s	1,7010ms 270ms	LCP ↓49% · TBT ↓73%
/txs	3.7s 2.2s	1,330ms 200ms	LCP ↓41% · TBT ↓85%
/tokens	3.3s 1.8s	510ms 190ms	LCP ↓45% · TBT ↓63%

02 /

파이레테크놀로지

2024.09 — 현재

STACK

- TypeScript
- React
- TailwindCSS
- wagmi · viem
- NestJS
- TanStack Query
- Jotai

모바일 WebView 거래 제품 구현

Pockie miniDApp · BTCFi Partners

각기 다른 호스트 환경과 파트너사에서 동일한 거래 화면을 노출·재사용하기 위해 서비스별로 BFF 계약과 연동 구조를 설계했습니다.

Pockie miniDApp — BiFi · Token Swap · BTCFi 거래 화면

- Pockie(크립토 자산 지갑 앱) WebView — **BFF 계약**을 기준으로 자산·거래 데이터를 처리하고 지갑 서명·전송은 호스트 지갑에 위임, Mobile App·Chrome Extension 두 환경에서 동일한 WebView 화면 재사용
- BiFi·Token Swap·BTCFi 서비스 3가지의 주요 거래 calldata 조회·vault 조회·상태 조회를 Pockie BFF API와 연결
- iOS·Android·Extension 및 앱 버전별 **feature flag**와 하위 호환 fallback을 **BFF config v2**에 구현 — 클라이언트 배포 없이 기능 노출 서버 제어

→ Mobile App·Chrome Extension 두 호스트에서 **BiFi·Swap·BTCFi 동일 화면 재사용** 구조 구현

BTCFi Partners — Hashport Wallet WebView 연동

- Bitcoin 담보 JPYC 스테이블코인 예치 상품 — 파트너사(Hashport Wallet) iOS·Android WebView에서 BTCFi Partners DApp 노출
- **postMessage** 이벤트를 협의·설계하고 **widget route**로 WebView 접근과 일반 웹 접근을 분리해 상품 로직 재사용

→ 출시 3개월간 일본 액티브 유저 **750명대**, 실제 입금 **1억 엔대** 운영 중

BTCFi BFF 구조 개선

- 클라이언트가 다수의 RPC·사내 API로 직접 분기 호출하던 구조 — 응답 순서에 따라 수치가 달라지는 **race condition** 발생, 캐싱·에러 핸들링 책임 경계도 불명확
- BFF에서 BiFi·Everdex status(예치·수익)와 BTCFi 온체인 자산(담보·LTV·청산가격)을 각각 집계해 **분리된 엔드포인트**로 재설계 — **race condition 제거** 및 자산 수치 일관성 확보

03 /

파이레테크놀로지

2024.09 — 현재

STACK

Claude Agent SDK

Codex app server

MCP

Slack

AI Agent 운영 및 AI 도구 활용

금쪽이 · 디자인 시스템 · 파이프라인

사내 AI Agent(금쪽이)의 런타임 구조 개선과 디자인 시스템 자동화 파이프라인 구현 두 가지로 구성됩니다.

금쪽이 — 컨텍스트 주입 구조 개선

- 메모리·프롬프트 주입 구조를 **core memory**와 **MCP on-demand search**로 분리하고, 변동 정보는 프롬프트 tail로 분리해 **캐시 적중률**에 영향을 주는 정적 prefix 안정화
- 메모리 주입 예산·우선순위 절단·관측 로그를 추가해 요청별 **컨텍스트 주입량을 통제 가능한 구조**로 정리

금쪽이 — 자동 리뷰 workflow와 품질 게이트

- Slack 기반 코드 리뷰 흐름에 리뷰 근거·**confidence verifier**(임계값 0.8, 별도 LLM 호출로 품질 검증) 추가
- CI 상태를 **30초** 간격으로 polling해 CI 통과·테스트·타입·severity 기준을 모두 만족한 경우 **자동으로 리뷰를 제출**
- 사람이 최종 확인하는 운영 원칙에 따라 **자동 머지는 구현하지 않음**

금쪽이 — MCP 도구 실행 구조 개선

- 기존: 새 request마다 등록된 MCP 서버 전체를 stdio로 spawn → 동시 요청 시 프로세스 메모리 경합·응답 저하
- lazy-proxy** 도입 — request 시작 시 연결 관리자만 준비, 특정 tool이 처음 호출될 때만 해당 서버를 spawn, request 종료 시 전체 정리

금쪽이 — Codex app server 전환과 보안 gate

- Claude Agent SDK 런타임을 **Codex app server** 기반으로 전환 — read/write tool 권한 구조 통합, dynamic tools 지원에 맞춰 도구를 자체 검색·활성화하는 구조로 재설계
- tool 실행 경로에 **3단계 보안 레이어**(활성화 검사 → bash 명령 안전성 검증 → write 작업 승인 정책) 구성, 외부 코드 실행과 환경 오염 가능 작업은 **workspace sandbox**로 격리

디자인 시스템 코드 생성 컨텍스트 정형화

- 디자인 시스템 컴포넌트 재사용 코드를 Figma MCP 컨텍스트로 정형화하기 위해 **Code Connect CLI** 도입 — 모델이 컴포넌트를 구현·재사용할 때 정확한 코드 스펙을 참조하도록 컨텍스트 주입
- 합성 컴포넌트(Tabs·Toast 등)는 동적 children 구조로 정적 매핑 불가 → **Figma Template V2 API**로 런타임에 인스턴스를 순회·재귀 실행해 실제 Figma 구성 기반 코드 생성

04 /

스마트마인드

2021 — 2024

STACK

React 18

Next.js

Vite

pnpm · Turborepo

Monaco Editor

ANTLR

React-Query · SWR

Recoil · Zustand

emotion · MUI

Playwright

ThanoSQL — 랜딩/콘솔 · Workspace

ThanoSQL은 정형·비정형 데이터 모두 SQL 쿼리를 사용해 AI 모델링 및 검색을 가능하게 한 통합 플랫폼. 랜딩/콘솔 페이지는 서비스 개요와 제품 진입 경로를 제공하며, Workspace는 JupyterLab 기반 노트북 환경(Lab)과 SQL 쿼리 편집기(Query Manager)를 제공하는 Data Lakehouse 서비스.

랜딩/콘솔 — 기여 내용

- 검색 엔진 크롤러가 콘텐츠를 인덱싱할 수 있도록 **Next.js SSR** 도입
- 별도 서버에 각각 배포된 ThanoSQL 기반 데모 프로젝트들을 iframe으로 통합해 Playground 단일 페이지에서 제공하도록 개발
- 메인 도메인의 쿠키에 사용자의 로그인 토큰을 저장·접근·삭제하기 위한 **Next.js API 개발**, **Google OAuth 로그인** 적용
- **Playwright e2e-test 도입**: Navigation check, Input form validation UI 테스트 코드 작성, Vercel Preview deployment에 대해 자동 검사하도록 **GitHub Actions** 설정

Workspace — Microfrontend 아키텍처

pnpm Workspace + Turborepo — 5 App 분리

File Manager · Query Manager · Lab · API · Main으로 App 단위 분리. 관심사 분리 및 Vercel 개별 배포.

Main(shell) → Lab — iframe embed

Main 앱이 온프레미스 JupyterLab 서버를 iframe으로 embed. postMessage로 인증 처리 완료 후 iframe 콘텐츠를 로드.

Query Manager · File Manager — module federation

@originjs/vite-plugin-federation으로 런타임 모듈 공유. 빌드 없이 원격 모듈 로드.

Workspace — 빌드·번들 최적화

- Workspace 패키지 매니저 변경(yarn → pnpm)과 turbo cache 설정으로 각 앱별 평균 빌드 시간 최대 **7분 → 1분** 단축
- vite rollup manualChunk, tree-shaking, dynamic import, React Suspense로 First Load **최대 40% 감소**

Workspace — 공통 인프라

- monaco-editor base로 사내 자체 개발된 SQL용 Query Editor 컴포넌트 개발
- Monorepo 공용 패키지 개발 — 공용 eslint, tsconfig, svgr-cli를 통한 아이콘 컴포넌트화, @types/ 패키지화

05 /

스마트마인드

2022 · PoC

STACK

React

Python

pandas

numpy

Facebook Ads API

Google Ads API

M.A.D.E PoC

광고 데이터 파싱·전처리 파이프라인

광고주와 프리랜서 마케터를 중개하고, 마케터들이 집행한 Google·Facebook 광고의 주요 성과 지표(CTR, CPC, CPM, Conversion rate)를 라이브 레포트로 제공하는 서비스.

기여 내용

- 연동된 마케터 계정의 광고 리포트 데이터를 파싱하여(Facebook Graph Ads API, Google Ads API) 자사 데이터 모델링 내용에 맞게 전처리하는 파이프라인 개발
- 메인 랜딩 웹페이지 개발

06 /

스마트마인드

2021 — 2022 · 데이터
바우처

STACK

React

JavaScript

Material-UI

FastAPI

Python

mssql

Docker

데이터 바우처 과제 — Steco SaaS · P2P-DC

드론 고장 분석 · 발전 예측량 · 잉여 전력 예측·실측 API

두 건의 데이터 바우처 과제로, Steco는 YOLOv5 열화상 분석 기반 드론 고장 분석과 발전 예측량(daily, long-term) 레포트 웹 서비스이고, P2P-DC는 잉여 전력 p2p 블록체인 거래 앱에 연결되는 잉여 전력 실제값·예측값 제공 API.

SteCo SaaS — 기여 내용

- JavaScript, React, Material-UI로 컴포넌트 구현·스타일 관리, Context API로 전역 상태 관리
- FastAPI·mssql로 API 구현, AI 팀 파인튜닝 PyTorch .pt 모델 파일 서빙 인프라 구성

P2P-DC — 기여 내용

- 외부 데이터 API에서 매일 cron으로 실측 데이터를 수집하고, AI 팀 예측 알고리즘을 실행해 실측·예측 데이터 파일을 저장하는 파이프라인 구성
- cron 실패 시 UI에서 직접 데이터 갱신을 트리거할 수 있는 수동 업데이트 API 개발 — 파이프라인 장애 시 운영 연속성 확보
- 해당 데이터를 차트용으로 제공하는 조회 API 개발, Docker 기반 배포 환경 구축